10. GPT 2

16 March 2026 14:02

question Answering
Machine translation
Reading comprehension
Summarization
} typically approached with Supervised learning on task specific datasets

→ They demonstrate that LM's learn these without any explicit supervision

All the mentioned tasks are still solved via next-token prediction

What changes :- → prompt format
→ Conditioning context

Language model capacity → essential for the success of zero-shot task transfer

↳ higher → increases its performance in a log linear fashion across tasks

↳ $a*(\log(\text{model-size}))+b$

demonstrating but steady gains as model grows

Paper's claim → next token prediction alone is sufficient to induce multitask behavior

GPT-2 → 1.5B parameter transformer

Structured prompt → induces task

Tasks emerge from Scale + data + Objective

→ paper says no task specific fine tuning → no supervised task labels ✓

but during training → Self supervised next-token prediction → no supervision at all ✗

Next token prediction
+ large model
+ large dataset
} emergent multi task system

When the trained model is tested on tasks like translation etc
Whether can it do them without task specific training
Zero shot task transfer (paper terminology)

They don't explicitly say for retraining
Shouldn't exist

system at that time narrow experts ✓ ✗ competent generalists

bottleneck mentioned (data set, objective) pairs

each pair requires
data collection
labels
training
evaluation
} this doesn't scale well

They wanted to connect
language model pretraining
+
Multi task learning
(language modeling alone may implicitly teach many tasks)

↳ no parameter updates or architecture changes downstream

Instead of
pretrain → fine tune
↓
pretrain → evaluate directly

☆ Reflections — Correction

① tasks require more context → input becomes longer
Mostly correct
But length increase not the main idea
Conditioning structure is

eg Simple LM training
Cat sat on the ___

Translation conditioning

Translate English to french
English: cat sat on the mat
French: ___

intuition about context is right but isn't the central claim of the paper

② GPT-1 had supervised fine tuning & GPT 2 removes that
→ Exactly

GPT-1 → stage 1 unsupervised LM training
stage 2 task specific fine tuning
each task required separate gradient updates

GPT-2 → Only unsupervised language modelling
☆ Tasks are solved using prompting only

Conditioning means model output depends on some provided variable

Approach

Language modeling is usually framed as unsupervised estimation from a set of examples $(x_1, x_2, \ldots, x_n)$ each composed of variable length sequences of symbols $(s_1, \ldots, s_n)$

language has natural sequential ordering → common to factorize the joint probabilities over symbols as product of conditional probabilities

$$p(x) = \prod_{i=1}^{n} p(s_i \mid s_1, \ldots, s_{i-1})$$

prob of whole sequence
token at position i
previous tokens
full sequence

allows tractable sampling from and estimation of $p(x)$ as well as any conditionals of the form $p(s_{n-k}, \ldots, s_n \mid s_1, \ldots, s_{n-k-1})$

Instead of modelling the probability of a full sentence directly
P("cat on the mat")
we factor x into sequential predictions
P(The)
P(Cat | The) → training = learning these conditional distributions

learning to perform a single task can be expressed in a probabilistic framework as estimating a conditional distribution $p(\text{output} \mid \text{input})$

A general system should be able to perform many tasks, even for the same input, it should condition not only the input but also the task to be performed

$p(\text{output} \mid \text{input}, \text{task})$

language provides a flexible way to specify tasks, inputs & outputs all as a sequence of symbols

Many tasks, even for same input, it should condition not only the input text also the task to be performed

$\downarrow$ $P(\text{output} \mid \text{input}, \text{task})$

Because everything is a sequence of symbols, task can be written as text

eg Translation

(translate to, spanish, English text, spanish text)

The task instruction becomes the part of input sequence

$P(\text{answer} \mid \text{task instruction} + \text{input text})$

Language provides a flexible way to specific tasks, inputs & outputs all as a sequence of symbols

Supervised objective is same as unsupervised but only evaluated on a subset of the sequence

global min of unsupervised objective is also the same for supervised case

The problem became "if we can optimize unsupervised objective to convergence"

preliminary experiments proved it but learning was slower than supervised approach

Moving towards systems that can learn from natural language directly

$\rightarrow$ Supervised task training

maximize $P(\text{output} \mid \text{input})$

Language model training

max imize $P(\text{next token} \mid \text{previous tokens})$

if the input and output are written as single sequence then objectives become equivalent

predicting answers is still next token prediction

so same LM objective can learn tasks implicitly

Input representations

Word level tokenization problem

dog
dogs
doggy

each variant becomes a new token $\rightarrow$ vocabulary explodes

Byte Pair Encoding —

Middle ground b/w above two

Basic idea

Start with characters

Merge frequently occurring pairs

Now words are encoded as sub-word pieces

lowest $\rightarrow$ low + est

lower $\rightarrow$ low + er

final vocabulary may contain low, er, est

eg corpus low
lower
lowest

initial tokens

l o w
l o w e r
l o w e s t

Capture most frequent pair

Suppose (l, o) appears frequently

Merge l o w
l o w e r
l o w e s t

next frequent pair (lo, w)

merge low
low e r
low e s t

BPE main advantages

$\rightarrow$ Vocabulary stays small

Instead of that dog's list we store something like

{ dog
s
gy

$\rightarrow$ Unknown words still work

$\rightarrow$ Sequences are shorter than characters

Most BPE implementations operate on Unicode characters

GPT-2 does BPE on bytes (UTF-8)

base alphabet is 256 possible bytes

$\rightarrow$ can represent any unicode text

$\rightarrow$ works across languages

$\rightarrow$ no complicated unicode preprocessing

~50,000 tokens (GPT-2 vocab size)

Architecture

Transformer Architecture

Mostly same as GPT-1

① They move layer norm to the input of each sub-block

Pre LN Transformer setup

$x \rightarrow$ layernorm $\rightarrow$ self attention $\rightarrow$ Residual

$x \rightarrow$ layernorm $\rightarrow$ feed forward $\rightarrow$ Residual

☆ Better training stability for transformers

② Residual scaling $\rightarrow$

Modified initialization accounts for accumulation on the residual path with model depth

We scale the wts of residual layers by $\frac{1}{\sqrt{N}}$, N = no of residual layers

☆ Residual networks accumulate activations

$x + f(x) + f_2(x) +$

We scale the weights of residual layers by $\frac{1}{\sqrt{N}}$, N = # of residual layers

* Residual networks accumulate activations
  $x + f(x) + f_2(x) +$
  network grows → activations may grow too large

  $W = w/\sqrt{N}$ → keeps variance stable as depth increases

Language provides a flexible way to specify tasks, inputs and outputs as a sequence of symbols
↳ itself becomes an interface to specify tasks

GPT-1 roughly looked like
h_1 = layernorm(x)
a = self attention(h_1)
$x = x + a$
h_2 = layer norm(x)
m = feed forward(h_2)
$x = x + m$

GPT-2
1. Layernorm moved to input of each sub block
2. An additional layernorm added after the final attention block

$x \to LN \to$ Attention + Residual
↓
LN ← added normalization
↓
feed forward

GPT1 → 12 layers
GPT2 XL → 48 layers → activation variance growth
unstable gradients

Model learns tasks as a side effect of modeling text distributions

They tested whether larger language models show better zero-shot ability?
↳ (Test scaling behavior)

Model size ↑
Perplexity ↓
Larger models fit distribution better
(Performance improves smoothly with scale
↳ Supports the claim
Capacity → multitask ability

They test many tasks without finetuning
Some tasks competitive with SOTA
Others → far behind finetuned systems
} without task training

→ Task ability improves with model size
Support the claim language modeling → emergent task ability

Generalization vs Memorization
Some test data overlaps with training data
↓ so models might memorize
usual percentage 1-6 %
Memorization exists but doesn't explain performance

* training perplexity ↓ → test perplexity ↓
↳ as models scale
The model is still underfitting → not overfitting

→ GPT-2 performs translations with zero parallel training data but weaker performance than real translation models

→ They tried summarization by writing a prompt
TL;DR
One of the first demonstrations of prompt based task control

→ Improves the previous LM baseline for commonsense reasoning

LLM + Large dataset → can perform many tasks without explicit supervision

* → Objective
Language modelling $p(x) = \prod p(s_i | s_{<i})$

→ Tasks can be written as text
eg (Question, context, answer)

→ Scaling unlocks Multitask Behavior
Model size ↑
task ability ↑

→ Zero shot task transfer
↳ Model performs tasks without task training

Unsupervised trained model can perform many tasks without fine-tuning

↳ correct high level takeaway

model not fully unsupervised

→ Self-supervised language modeling

↳ emergent multitask behavior

Before → LM → features for downstream tasks

After this paper → LM → performs the tasks itself

During training the model sees the entire sequence at once

eg The cat sleeps

The model predicts $P(\text{The})$

$P(\text{cat} \mid \text{The})$

$P(\text{sleep} \mid \text{The cat})$

Best all predictions are computed in parallel during the forward pass

because of teacher forcing

input tokens = ground truth

Model never feeds its own predictions back during training

Without teacher forcing

↳ Model predicts wrong token early

↓

future predictions collapse

↓

training signal becomes unstable

Inference

→ we don't know future tokens

→ must generate one token at a time

i. Start with prompt

ii. run forward pass (model outputs logits

iii Choose next token

iv Append to input

v Run model again

(Auto regressive generation)

Why sampling matters?

if we choose highest probability token always → generation becomes repetitive

So we sample from probability distribution

Two techniques

i Temperature (modifies logits before softmax)

$$P_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_i/T)}$$

logits

temperature

Temperature

↳ is essentially scaling the logits

an effective way to control the entropy of output distribution

$T<1$ Sharper distribution

$T=1$ Original probabilities

$T>1$ Flatter distribution

ii Top K sampling

→ instead of sampling from all tokens

We restrict to K most probable tokens

All other tokens are removed

Then probabilities are renormalized